Aran Learn 8: Web APIs and Proxies

For: everyone. Inbathamizhan and Aasif build the gateway (roadmap item 1); Bharadhwaj and Arun build the AI gateway (item 6). **Time:** about 6 hours with practice. **Before this:** Learn 1 (Python, virtual environments, git) and Learn 2 section on Aadhaar and Verhoeff.

So far Aran has run as a script over log files. A bank's data does not sit in files waiting for us. It moves between programs as web requests, many thousands a second. To protect it "before it is stored or shared", Aran has to sit in the path of those requests. This doc explains what a web API is, how to build one in Python, and how to build a proxy that protects every response passing through it. It ends with an AI gateway that hides identifiers from a chatbot.

Every output shown below was produced by running the code in this doc.

1. HTTP in one page

When a mobile banking app shows your balance, it sends a **request** over the network to the bank's server and gets a **response** back. The rules for that conversation are **HTTP** (HyperText Transfer Protocol). **HTTPS** is the same thing inside an encrypted TLS tunnel (Learn 3), so nobody on the network can read it.

A request has four parts:

Part	Example	Meaning
Method	`GET`	What you want to do. `GET` reads, `POST` creates, `PUT` replaces, `DELETE` removes
Path	`/customers/C1001`	Which thing you mean
Headers	`Authorization: Bearer eyJ...`	Extra facts: who you are, what format you send
Body	`{"amount": 500}`	The data, for `POST` and `PUT` (usually empty for `GET`)

A response has a **status code**, headers and a body:

Code	Meaning	When you see it
200 OK	Worked	A normal read
201 Created	Worked, something new was made	A transfer was accepted
400 / 422	Your request was wrong	Missing or badly typed field
401 / 403	Not logged in / not allowed	Bad or missing token
404	No such thing	Unknown customer ID
500 / 502	Server broke / the server behind it broke	Bugs, outages

Codes starting with 2 mean success, 4 means the caller made a mistake, 5 means the server did.

An API (Application Programming Interface) is a set of request paths that programs, not people, are meant to call. A bank has hundreds of internal APIs: KYC, payments, statements, loans, the call centre tool. Each one is a place where identifiers move, and so a place where they leak.

2. JSON, the language of API bodies

Almost every modern API sends its body as **JSON** (JavaScript Object Notation). It has only six kinds of value: objects `{...}` with named fields, lists `[...]`, strings `"..."`, numbers, `true`/`false` and `null`.

```
{
  "name": "Priya Raman",
  "aadhaar": "726018159082",
  "mobile": "6166131860",
  "accounts": [{"type": "savings", "balance": 15230.50}]
}
```

In Python, `json.loads(text)` turns this text into dicts and lists, and `json.dumps(obj)` turns them back.

Why JSON matters for detection. The field name is free evidence. A 10-digit number under a key called `mobile` is almost certainly a mobile number, while the same number in a log line with no words around it is the weak spot Aran misses 32% of the time today. When Aran reads JSON, it should pass the key name to the detector as context. Roadmap item 3 does exactly this.

3. Build an API with FastAPI

FastAPI is a Python library for writing APIs. You write ordinary functions, and it handles the HTTP parts, checks the body's types, and writes documentation pages for you.

Set up in your Aran working copy (on Windows, use Git Bash as in Learn 1):

```
cd aran
mkdir -p practice/learn8 && cd practice/learn8
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install fastapi uvicorn httpx cryptography
```

3.1 A fake bank API

Save as `bank_api.py`. The values are generated test values that pass their checksums but belong to no one.

```
# bank_api.py: a fake bank API that returns raw customer data (the thing Aran must protect)
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Fake bank API")

CUSTOMERS = {  # generated test values: checksum-valid, belong to no one
    "C1001": {"name": "Priya Raman", "aadhaar": "726018159082", "mobile": "6166131860",
              "pan": "HUUGB9960H", "card": "4824628194821991", "city": "Madurai"},
}

class Transfer(BaseModel):
    from_account: str
    to_upi: str
    amount: float
```

```
@app.get("/customers/{cid}")
def get_customer(cid: str):
    if cid not in CUSTOMERS:
        raise HTTPException(status_code=404, detail="no such customer")
    return CUSTOMERS[cid]

@app.post("/transfers", status_code=201)
def make_transfer(t: Transfer):
    return {"status": "accepted", "amount": t.amount, "to_upi": t.to_upi}
```

Line by line:

- `@app.get("/customers/{cid}")` says "when a GET request arrives for a path like `/customers/C1001`, call this function with `cid="C1001"`".
- Returning a dict makes FastAPI send it as JSON with status 200.
- `raise HTTPException(status_code=404, ...)` sends a 404 instead.
- `Transfer` is a **Pydantic model**: it declares what a valid body looks like. If a caller sends `{"amount": "lots"}`, FastAPI rejects it with 422 before your function runs. Checking input at the edge like this is a basic security habit.

3.2 Run it and call it

```
uvicorn bank_api:app --port 8001
```

`uvicorn` is the web server that runs a FastAPI app. Leave it running, open a second terminal, and call it:

```
curl http://127.0.0.1:8001/customers/C1001
```

```
{"name":"Priya
Raman","aadhaar":"726018159082","mobile":"6166131860","pan":"HUUGB9960H","card":"482462819482
1991","city":"Madurai"}
```

Also open `http://127.0.0.1:8001/docs` in a browser. FastAPI wrote an interactive page where you can try every path. That page is handy for a demo, and in production it should be switched off or protected, because it is a map of your API for an attacker too.

Every number, name and identifier above just crossed the network in plain JSON. Anything that receives this response, such as a partner app, a log collector or a monitoring tool, now holds a raw Aadhaar number.

4. Proxies: standing in the middle

A **proxy** is a program that receives requests meant for another server, passes them on, and passes the answers back. Because every byte goes through it, it can inspect and change them.

- A **forward proxy** sits next to the clients, for example an office network's web filter.
- A **reverse proxy** sits in front of servers. Callers think they are talking to the bank API, but they are talking to the proxy, which talks to the API. Nginx, Envoy and cloud API gateways are reverse proxies.

Aran is a reverse proxy that protects data on the way through:

```
 partner app  ──request──▶  Aran proxy  ──request──▶  bank API
              ◀─protected──             ◀───raw─────
                                  │
                                  └─ detects, tokenises, counts the flow
```

The bank API does not change at all. That is the main selling point to a bank: no team has to rewrite its service.

4.1 Build the Aran proxy

Save as `aran_proxy.py`. It reuses Aran's own core: `verhoeff_valid` to decide whether a 12-digit number is a real Aadhaar, and `cc_token_aadhaar` to replace it with a checksum-complement token (Learn 2 and Learn 3).

```
# aran_proxy.py: a reverse proxy that protects every response before it leaves
import json, os, re, sys, time
import httpx
from fastapi import FastAPI, Request, Response

sys.path.insert(0, os.environ.get("ARAN_RESEARCH", "../../research"))
from aran_core import FF1, cc_token_aadhaar, verhoeff_valid  # noqa: E402

UPSTREAM = os.environ.get("UPSTREAM", "http://127.0.0.1:8001")
KEY = os.environ.get("ARAN_KEY", "demo-key-change-me-0123456789abc").encode()[:32]
ff1 = FF1(KEY)
SINK = "partner-app"                       # who receives these responses
AADHAAR = re.compile(r"(?<!\d)\d{12}(?!\d)")
MOBILE = re.compile(r"(?<!\d)[6-9]\d{9}(?!\d)")

def protect(text: str) -> str:
    def aadhaar(m):
        v = m.group()
        return cc_token_aadhaar(ff1, KEY, v, SINK) if verhoeff_valid(v) else v
    text = AADHAAR.sub(aadhaar, text)
    return MOBILE.sub(lambda m: "XXXXXX" + m.group()[-4:], text)

app = FastAPI(title="Aran proxy")
client = httpx.AsyncClient(base_url=UPSTREAM, timeout=5.0)

@app.api_route("/{path:path}", methods=["GET", "POST", "PUT", "DELETE"])
async def forward(path: str, request: Request):
    t0 = time.perf_counter()
    up = await client.request(request.method, "/" + path, content=await request.body(),
                              headers={"content-type": request.headers.get("content-type",
"application/json")})
    body = protect(up.text)
    ms = (time.perf_counter() - t0) * 1000
    return Response(content=body, status_code=up.status_code, media_type="application/json",
                    headers={"x-aran-ms": f"{ms:.2f}"})
```

The new ideas:

- `"/{path:path}"` matches **every** path, so one function forwards everything.

- `httpx.AsyncClient` is a Python HTTP client. The proxy uses it to call the real API. `async/await` let one proxy process wait on many slow upstream calls at once instead of one at a time.
- `timeout=5.0`: never wait forever on the server behind you. A proxy without timeouts can be frozen by one slow service.
- The status code from the bank API is passed through unchanged, so a 404 stays a 404.
- `x-aran-ms` is a custom response header reporting how long the proxy took. Banks will ask about added delay, so measure it from day one.

Run the bank API on port 8001 (section 3.2) and the proxy on port 8000:

```
uvicorn aran_proxy:app --port 8000
curl http://127.0.0.1:8000/customers/C1001
```

4.2 What came out

Calling the bank directly and then through the proxy:

```
direct:  200 {"name":"Priya
Raman","aadhaar":"726018159082","mobile":"6166131860","pan":"HUUGB9960H","card":"482462819482
1991","city":"Madurai"}
proxied: 200 {"name":"Priya
Raman","aadhaar":"212343716312","mobile":"XXXXXX1860","pan":"HUUGB9960H","card":"482462819482
1991","city":"Madurai"}
token passes Verhoeff? False
404: 404 {"detail":"no such customer"}
post: 201 {"status":"accepted","amount":500.0,"to_upi":"priya@oksbi"}
bad body: 422
median 3.85 ms, p95 4.84 ms
```

The Aadhaar became `212343716312`. It still has 12 digits, so the partner app's database and screens keep working, but it fails Verhoeff, so any scanner can prove it is a token and not a leaked number. The mobile is masked. Errors and the 201 pass through unchanged, and a badly typed body is still rejected with 422 by the bank API.

The last line is the delay per request measured on one laptop core, with both apps in one process (the test harness in section 4.3). **Median** means half the requests were faster than this; **p95** means 95% were faster. Banks quote p95 or p99, not the average, because the slow requests are the ones customers notice.

What this toy leaves raw on purpose: the name, PAN and card. Practice P2 and P3 fix that. The real gateway (roadmap item 1) calls the full `AranFirewall` from `research/firewall/aran_firewall.py`, which handles every type.

4.3 Test without starting servers

Starting two servers for every test is slow. FastAPI's `TestClient` calls an app directly in memory, and `httpx.ASGITransport` lets the proxy call the bank app the same way. Save as `test_proxy.py`:

```
import httpx
from fastapi.testclient import TestClient
import bank_api, aran_proxy
from aran_core import verhoeff_valid

aran_proxy.client = httpx.AsyncClient(transport=httpx.ASGITransport(app=bank_api.app),
```

```
base_url="http://bank")
proxy = TestClient(aran_proxy.app)

def test_aadhaar_is_tokenised():
    d = proxy.get("/customers/C1001").json()
    assert d["aadhaar"] != "726018159082"
    assert len(d["aadhaar"]) == 12 and not verhoeff_valid(d["aadhaar"])

def test_errors_pass_through():
    assert proxy.get("/customers/NOPE").status_code == 404
```

Run with `python -m pytest test_proxy.py`. These are the kind of tests the gateway must have before anyone demos it.

5. Securing the proxy itself

A proxy that sees every raw identifier is the most valuable thing in the network to an attacker. Before it goes anywhere near real data:

Risk	What to do
Someone reads traffic on the wire	HTTPS on both sides; never plain HTTP outside a laptop demo
Anyone can call it	Require a token in the `Authorization` header and check it on every request
The proxy's own logs leak data	Never log request or response bodies; log only path, status, time and Aran's counts
Keys in the code	Read keys from the environment or a key manager (Learn 3). The default key above is for the demo only
Huge bodies freeze it	Set a maximum body size; reject bigger requests with 413
Slow upstream freezes it	Timeouts on every upstream call (we used 5 s)
Aran crashes	Decide in advance: **fail closed** (return 503, nothing leaves) or **fail open** (pass raw data through). For identifiers, fail closed

The last row is a real design decision, and a good one to raise with judges: Aran chooses to break a request rather than leak a raw Aadhaar.

6. Logs arrive as requests too

Log shippers such as Fluent Bit and Vector send batches of log lines to a collector over HTTP. Aran can be that collector: accept a batch, protect each line, and pass the clean batch on to the log platform.

```
from pydantic import BaseModel

class LogBatch(BaseModel):
    source: str          # which service wrote these lines
    lines: list[str]

@app.post("/logs")
def ingest(batch: LogBatch):
    clean = [protect(line) for line in batch.lines]
```

```
    # forward `clean` to the log platform here
    return {"accepted": len(clean)}
```

The `source` field is what the immune layer needs (Learn 6): each batch becomes a data flow from `source` to the log platform, counted per hour.

7. The chat API format, and an AI gateway

Bank staff paste customer data into chatbots. Roadmap item 6 puts Aran in front of the chatbot so the AI provider never receives a raw identifier.

7.1 The format

Most AI providers and open-source model servers accept the same request shape, first made popular by OpenAI's Chat Completions API:

```
POST /v1/chat/completions
{
  "model": "some-model",
  "messages": [
    {"role": "system", "content": "You are a bank assistant."},
    {"role": "user", "content": "Customer 726018159082 wants a loan."}
  ]
}
```

`messages` is the conversation so far. `system` sets the assistant's behaviour, `user` is what the person typed, and earlier `assistant` turns are the model's past replies. The answer comes back in `choices[0].message.content`. Because so many tools speak this format, a gateway that accepts it can sit in front of almost any model without changing the tools that call it.

7.2 Typed tokens instead of numbers

For a log, a checksum-complement token is ideal. For a chatbot, a **typed token** such as `<AADHAAR_1>` is better: the model can still reason ("the customer's Aadhaar is verified") without ever seeing the number, and the type tells it what kind of thing it is.

Save as `ai_gateway.py`:

```
# ai_gateway.py: an OpenAI-compatible endpoint that never lets raw identifiers reach the
model
import re
from fastapi import FastAPI, Header, HTTPException

PATTERNS = {"AADHAAR": re.compile(r"(?<!\d)\d{12}(?!\d)"),
            "PAN": re.compile(r"\b[A-Z]{5}\d{4}[A-Z]\b")}

def tokenise(text, vault):
    for typ, pat in PATTERNS.items():
        def swap(m):
            for tok, val in vault.items():          # same value, same token
                if val == m.group():
                    return tok
            tok = f"<{typ}_{len(vault) + 1}>"
            vault[tok] = m.group()
```

```
            return tok
        text = pat.sub(swap, text)
    return text

def detokenise(text, vault):
    for tok, val in vault.items():
        text = text.replace(tok, val)
    return text

def fake_model(messages):                          # stands in for the real LLM call
    last = messages[-1]["content"]
    return f"I have noted the request. Summary: {last}"

SEEN_BY_MODEL = []                                 # for the demo only: what the provider
received
app = FastAPI(title="Aran AI gateway")

@app.post("/v1/chat/completions")
def chat(body: dict, x_user_role: str = Header("agent")):
    vault = {}                                      # lives for this one request only
    safe = [{"role": m["role"], "content": tokenise(m["content"], vault)} for m in
body["messages"]]
    SEEN_BY_MODEL.append(safe)
    reply = fake_model(safe)
    if x_user_role == "kyc-officer":                # only this role may see real values back
        reply = detokenise(reply, vault)
    return {"object": "chat.completion", "model": body.get("model", "demo"),
            "choices": [{"index": 0, "message": {"role": "assistant", "content": reply},
"finish_reason": "stop"}]}
```

Sending the loan message once as a normal agent and once as a KYC officer gave:

```
model saw:    Customer <AADHAAR_1> with PAN <PAN_2> wants a loan. Aadhaar <AADHAAR_1> is
verified.
agent gets:   I have noted the request. Summary: Customer <AADHAAR_1> with PAN <PAN_2> wants
a loan. Aadhaar <AADHAAR_1> is verified.
officer gets: I have noted the request. Summary: Customer 726018159082 with PAN HUUGB9960H
wants a loan. Aadhaar 726018159082 is verified.
```

Three things to notice:

1. The same Aadhaar appearing twice became the same token both times, so the model still understands it is one customer.
2. The vault lives only for one request. Nothing about the real numbers stays in the gateway afterwards.
3. Only the officer got the real values back. This is the **detokenise** step, and in Aran every detokenise call is logged per caller, because an insider who detokenises and then leaks is the hardest attack in the research doc's critical review.

A deliberate flaw to fix. Here the role comes from a header the caller sets, so anyone can send `x-user-role: kyc-officer`. In the real gateway the role must come from a verified login token (practice P5). Never trust a permission the client states about itself.

To call a real model instead of `fake_model`, replace it with an `httpx` POST of `safe` to the provider's `/v1/chat/completions` URL with your API key in the `Authorization` header. Keep the key in an environment variable, never in the code or the repo.

8. Practice

- [ ] **P1.** Run `bank_api.py` and `aran_proxy.py` as in sections 3.2 and 4.1 and reproduce the output in 4.2 with `curl`. Then open `/docs` on both.
- [ ] **P2.** Protect the card number in the proxy using `cc_token_card` from `research/firewall/aran_firewall.py` and `luhn_valid`. Add a test that the token fails Luhn.
- [ ] **P3.** Use the JSON key as context: parse the body with `json.loads`, and if a key is `mobile`, `phone` or `contact`, mask the value even if it does not start with 6 to 9. Write one test that passes and one that shows a case it still misses.
- [ ] **P4.** Add a `/logs` endpoint (section 6) and count lines per `source` per hour in a dict. Print the counts every 100 batches.
- [ ] **P5.** Replace the `x-user-role` header with a check of a shared secret token in `Authorization: Bearer ...`, read from an environment variable. Return 401 when it is missing or wrong.
- [ ] **P6.** Make the proxy fail closed: wrap `protect` in `try/except` and return 503 with no body if it raises. Prove it with a test that makes `protect` raise.
- [ ] **P7.** Measure median and p95 delay for 1,000 requests with and without the proxy, and write down the difference. This is the number for the demo slide.

9. Further reading

- MDN Web Docs, "An overview of HTTP" and "HTTP response status codes" (free, developer.mozilla.org).
- FastAPI tutorial, "First steps" to "Request body" (fastapi.tiangolo.com/tutorial).
- HTTPX documentation, "Async support" and "Transports" (www.python-httpx.org).
- OWASP API Security Top 10, 2023 edition (owasp.org), especially API3, Broken Object Property Level Authorization (which absorbed the older "excessive data exposure" item): an API returning more fields than the caller needs is exactly the problem Aran solves.
- Envoy proxy, "What is Envoy" (envoyproxy.io), for how real banks run reverse proxies at scale.
- OpenAI API reference, "Chat Completions" (platform.openai.com/docs/api-reference/chat), for the format in section 7.